

AI CIRCUIT BREAKER

End-to-End Flow for the SWaT Testbed

Morphism-Grounded Compositional Assurance for Autonomous AI Systems

Document Type	Technical Design — End-to-End Flow Specification
Date	April 2026
Domain	Critical Infrastructure Security / Trustworthy AI / ICS
Application	SWaT (Secure Water Treatment) Testbed — Attack Detection
Framework	DARPA CLARA: Morphism-Grounded Compositional Assurance
Focus Area	Attack Detection Effectiveness, Per-Agent Trust, Compliance-on-Demand

1. Executive Summary

This document specifies a complete end-to-end flow for deploying the AI Circuit Breaker and Trust Engine against the SWaT (Secure Water Treatment) testbed — a widely-used research platform for cyber-physical system security research. The flow directly addresses CLARA program objective (b): assessing the effectiveness of attack detection algorithms on a real industrial control system benchmark dataset.

The architecture demonstrates the circuit breaker as a reusable, domain-agnostic module that can be independently instantiated and tuned for EACH of the six SWaT PLCs (Process Logic Controllers), while simultaneously computing collective compositional trust scores across the full multi-agent system. This dual individual/collective model directly reflects the Morphism Composition Theorem: $D_{s_total} \geq \max(D_{s_i})$, ensuring that the overall system's assurance can never exceed its weakest-link agent — a formal guarantee, not an assumption.

Three enforcement actions are specified: Compliance-on-Demand (policy assertion without interruption), Freeze (unauthorized activity suspension), and Halt (hard stop for agents that exceed control boundaries). The trust evaluation pipeline integrates the Semantic Anomaly Score (S_a), Contextual Relevancy Index (C_r), Mean Time Between Hallucinations (MTBH), and the Human-AI Calibration Coefficient (K_{trust}) into a unified verdict that drives these enforcement actions deterministically.

Design Principle — The Deterministic Veto

The SWaT sensor reading constitutes ground truth. No AI classification assertion may be acted upon operationally if its Intent Delta exceeds the Sensor Ground Truth Tolerance. The circuit breaker is not advisory — it is a deterministic veto with absolute authority over all autonomous AI actions across all six PLC agents.

2. SWaT Testbed Overview and Agent Topology

The SWaT testbed (iTrust, Singapore University of Technology and Design) is a fully operational scaled-down water treatment plant comprising six sequential stages, each controlled by a dedicated PLC agent. The testbed has been extensively instrumented to provide ground truth attack labels across 41 distinct cyber-physical attack scenarios, making it the gold standard for evaluating ICS attack detection algorithms.

2.1 Six-Stage Agent Architecture

Stage	PLC Agent	Key Sensors	Key Actuators	Physical Process
P1	PLC-1 (Raw Water)	LIT101 (level)	MV101, P101, P102	Raw water intake & storage tank
P2	PLC-2 (Chem Dosing)	FIT201, AIT201-203	P201–P208, MV201	Chemical dosing (NaCl, HCl, NaHSO3)

P3	PLC-3 (Dechlorination)	DPIT301, FIT301, LIT301	MV301, P301, P302	Chemical dechlorination & UF feed
P4	PLC-4 (Ultrafiltration)	FIT401, LIT401, P401-P404	UV401	Ultrafiltration membrane treatment
P5	PLC-5 (RO Feed)	AIT501-504, PIT501-503	P501, P502	Reverse Osmosis high-pressure feed
P6	PLC-6 (Clean Water)	FIT601, LIT601	P601, P602, P603	Treated water storage & distribution

2.2 SWaT Attack Dataset Profile

The SWaT dataset contains 7 days of normal operation followed by 4 days of attack scenarios. It comprises approximately 946,722 data points (at 1 Hz) across 51 process variables. The 41 labeled attack scenarios span single-point sensor manipulation, single-point actuator manipulation, and coordinated multi-point attacks targeting multiple PLCs simultaneously.

Attack Category	Count	Description	CB Relevance
Single-point sensor spoofing	16	False sensor readings injected at one point	Per-agent S_a trigger
Single-point actuator command	7	Unauthorized actuator state change	Per-agent C_r trigger
Multi-point coordinated	13	Simultaneous manipulation across stages	Collective D_s_total breach
Replay attack	3	Historical normal data replayed during attack	MTBH + drift detection
Other / compound	2	Complex attack vectors	Full CB trip + Halt

3. Morphism-Grounded Compositional Assurance Architecture

The CLARA framework grounds all trust assertions in morphism theory. Every SWaT PLC agent is formally modeled as a Wymore five-tuple $Z = (S, I, O, N, R)$, where S is the set of physical states (sensor readings), I is the set of inputs (external commands), O is the set of outputs (actuator actions), N is the set of internal state transitions, and R is the transition relation. The morphism $h: Z_{ai} \rightarrow Z_{real}$ is the formal object connecting the AI's learned representation to physical reality.

3.1 Morphism Instruments per SWaT Agent

Instrument	Formula	SWaT Physical Interpretation	CB Threshold
Structural Distance (D_s)	$1 - (I \cdot N_o / I \cdot N_o)$	How far AI's intent embedding deviates from valid SWaT ontological state space (e.g., pump-on with closed valve)	$D_s < 0.15 \rightarrow \text{pass}; \geq 0.15 \rightarrow \text{Semantic Veto}$
Behavioral Distance (D_b)	$\max_t y_{ai}(t) - y_{real}(t) $	Max deviation between AI-commanded actuator state and confirmed physical sensor ground truth (e.g., LIT101 delta)	$D_b \leq \text{sensor tolerance (stage-specific)}$
Composition Bound (D_{s_total})	$\geq \max(D_{s_i})$	Collective system structural distance: no higher than worst single-agent structural drift	Breach triggers collective Freeze
Behavioral Sum (D_{b_total})	$\leq \text{sum}(D_{b_i})$	Worst-case total behavioral divergence across all 6 PLC agents	Breach triggers system-wide Halt
Cosine Sentinel (d_{cos})	per-inference runtime	Per-inference cosine alignment monitor on AI intent vs. ontology centroid	$d_{cos} < 0.85 \rightarrow \text{breaker pre-alert}$

3.2 Four-Layer Architectural Stack for SWaT

Layer	SWaT Function	Ground Truth Source	Failure Mode
Data Layer	Raw sensor acquisition (LIT, FIT, AIT, PIT, DPIT, MV, P states at 1Hz)	Physical electrochemical/mechanical sensor at process interface	Sensor spoofing, replay injection, noise
Algorithmic Layer	Digital feature extraction: RR-intervals $\rightarrow$ SWaT analog: flow differentials, pressure	Deterministic DSP algorithms over raw readings	Temporal misalignment, compression artifacts

	gradients, chemical concentration trends		
LLM / AI Layer	Anomaly classification (CUSUM, IF, LSTM, GNN), attack type prediction, risk scoring per PLC	Probabilistic inference over learned feature distributions	Super-Huberous assertions: high-confidence normal verdict during active attack
Circuit Breaker Layer	Continuous delta evaluation, ontology verification, safe state enforcement per PLC and collectively	Deterministic veto rules + W3C RDF/OWL SWaT process ontology	Misconfigured thresholds, incomplete ontology for novel attack vectors

4. End-to-End Flow: Eight Phases

The end-to-end flow is organized into eight sequential phases, spanning pre-deployment design-time activities through runtime enforcement and recursive governance. Each phase maps directly to one or more subsystems of the Trustworthiness Engineering model. The flow is designed to be inserted as a reusable module around any attack detection algorithm deployed on the SWaT testbed.

PHASE 0 | DESIGN TIME — SWaT Ontology & Semantic Guardrails

Knowledge Engineering Subsystem: Encoding ICS Impossible States

Before any agent is deployed, the SWaT process domain is encoded into a W3C RDF/OWL ontology that defines the Immutable Truth of the water treatment process. This ontology functions as the non-negotiable constraint set that governs every runtime AI assertion.

0.1 SWaT Ontology — Impossible States Encoding

Category	Formal Expression	Physical Meaning
Physical Impossibility	LIT101 > 1200mm AND P101=OFF AND MV101=OPEN	P1 tank overflow with intake open, pump off
Chemical Impossibility	AIT201 > 2.0 S/m AND P201=OFF	Chlorine sensor reads dosing without pump active
Flow Impossibility	FIT301 > 0.5 m ³ /h AND MV301=CLOSED	Flow registered with valve physically closed
Pressure Impossibility	PIT501 > 8.5 bar AND P501=OFF	RO high-pressure with pump off
Level Impossibility	LIT601 < 250mm AND P601=ON	Distribution pump running on near-empty tank
Cascade Impossibility	FIT401 > 0 AND LIT301 < 200mm AND P301=OFF	UF feed active with dechlorination tank empty

0.2 Knowledge Fabric Technology Stack

Component	Technology	SWaT Role
Domain Ontology	OWL API + Apache Jena Fuseki	SWaT process ontology: stages, sensors, actuators, valid state transitions encoded in OWL 2 EL++
Logic Reasoner	ProbLog2 (KU Leuven)	Bayesian LP inference: P(morphism_holds evidence); computes posterior over composition verdict
Vector Database	FAISS (Meta)	Semantic embeddings of historical SWaT normal/attack

		states; ANN for D _s nearest-neighbor retrieval
SPC Engine	Custom Python/scipy	Statistical Process Control with Western Electric rules + CUSUM for per-sensor drift detection
Provenance	PROV-O via rdflib	Queryable audit trail: every CB decision linked to sensor readings and ontological nodes verified
Negative Exclusion DB	Vector DB (flagged partition)	Attack-epoch embeddings stored as exclusion partition; purged from training queues (SOP-03)

PHASE 1 | PRE-DEPLOYMENT — SOP-01 Semantic Handshake

Initialization Gate: No Agent Operates Autonomously Until This Passes

Before any PLC AI agent is permitted to act autonomously on the SWaT testbed, it must complete the Semantic Handshake — a structured verification sequence confirming that the agent's internal model is consistent with the SWaT ontology and that sensor precision satisfies the Rule of Ten prerequisite.

SOP-01 Step	Action	Pass Criterion	Fail Consequence
Step 1.1	Ping Knowledge Graph — verify ontology version and coverage for assigned stage	All 6 stage sub-graphs reachable; ontology version $\geq$ current	Agent BLOCKED from deployment
Step 1.2	Sensor Precision Check — verify Rule of Ten for each sensor in agent scope	Sensor resolution $\leq 0.1 \times$ operational tolerance (e.g., LIT101 resolution $\leq 0.1\text{mm}$ for 1mm tolerance)	Agent BLOCKED; maintenance alert raised
Step 1.3	Morphism Certificate Generation — compute initial D _s for agent's embedding vs. ontology centroid	D _s _initial < 0.10 (below Semantic Veto threshold)	Agent requires retraining before deployment
Step 1.4	SHACL Compatibility Validation — verify agent's input/output schema against adjacent agents	All SHACL shapes satisfied for composition interface	Interface redesign required
Step 1.5	Baseline Trust Metrics Collection — establish MTBH baseline from historical data	MTBH baseline recorded; K _{trust} initialized = 1.0	Manual override required

Step 1.6	Human Trustee Sign-Off — operator reviews morphism certificate and approves autonomous mode	Trustee signature recorded in PROV-O audit trail	Agent remains in supervised mode
----------	---	--	-------------------------------------

PHASE 2 | RUNTIME INGESTION — Sensor Data & Ground Truth Vector

Metrology Flow: Real-World Sensors → Context Subsystem → Ground Truth Vector

At runtime, each PLC agent ingests a continuous stream of sensor telemetry at 1 Hz. The Context Engineering Subsystem transforms raw readings into a structured Ground Truth Vector (S_gt) that the circuit breaker uses as its deterministic anchor. Simultaneously, the AI agent produces its Situational Context Model (C_m) — its internal belief state about the current process condition.

2.1 Ground Truth Vector Construction

Vector Component	SWaT Source Variables	Processing	CB Role
Physical State Vector S_gt	LIT*, FIT*, AIT*, PIT*, DPIT* (all active sensors)	Real-time normalization to [0,1] per operational range; timestamp alignment	Absolute authority; CB cannot override
Actuator State Vector A_gt	MV*, P* (all active actuators: OPEN/CLOSED, ON/OFF)	Boolean encoding; cross-check against PLC ladder logic expected state	Defines valid action space for AI agent
Cross-Stage Flow Consistency	FIT301→FIT401 mass balance; LIT101→FIT201 level-flow	Mass/volume conservation check across stage boundaries	Detects cascade manipulation attacks
Environmental Context	System uptime, current SPC control limits, CUSUM accumulators	Rolling window statistics; Western Electric rule flags	Provides drift and trend signals to CB
AI Situational Model C_m	Agent's internal state belief; predicted next state	Generated by AI agent's classifier/predictor module	Compared to S_gt for Intent Delta computation

2.2 Data Flows

Strategic Flow: Human Intent → Knowledge Subsystem → Strategic AI → Proposed Action Vector

↳ SWaT: Operator policy → ontology-constrained → PLC agent → actuator command candidate

Metrology Flow: Real-World Sensors → Context Subsystem → Ground Truth Vector (S_gt)

↳ SWaT: Physical process readings → feature extraction → deterministic ground truth

Validation Flow: Proposed Action Vector + S_gt → Circuit Breaker (Delta Analysis)

↳ SWaT: AI intent vs. physical reality → morphism instruments → verdict

Governance Flow: Breaker Trip/Success → Data Exclusion Filter → Recursive Learning → KB Update

↳ SWaT: CB outcome → model governance → prevent re-learning attack patterns

PHASE 3 | PER-AGENT TRUST ASSESSMENT — Individual Circuit Breaker Module

Each PLC Agent Gets an Independently Tuned Circuit Breaker Instance

This is the core reusability demonstration. A single circuit breaker module template is instantiated six times — once per PLC agent. Each instance shares the same architectural structure but is configured with stage-specific thresholds, sensor tolerances, and ontological constraints. The module is fully self-contained and can be added, removed, or reconfigured without modifying adjacent agent instances.

3.1 Per-Agent Module Configuration

PLC Agent	Stage Ontology Scope	Key Threshold (D_b)	CUSUM k-Factor	Safe State Action
PLC-1 (P1)	Raw water intake physics; tank level bounds 250–1200mm	±15mm (LIT101)	k=0.5, h=5	Close MV101; disable P101/P102
PLC-2 (P2)	Chemical dosing stoichiometry; flow-concentration correlation	±0.05 S/m (AIT201)	k=0.5, h=4	Disable dosing pumps P201–P208
PLC-3 (P3)	Dechlorination kinetics; differential pressure limits	±0.02 bar (DPIT301)	k=0.6, h=5	Close MV301; halt P301/P302
PLC-4 (P4)	UF membrane pressure envelope; backwash sequencing	±0.05 m³/h (FIT401)	k=0.5, h=5	Open backwash valve; disable UV401
PLC-5 (P5)	RO high-pressure physics; permeate/reject chemistry	±0.3 bar (PIT501)	k=0.4, h=4	Halt P501/P502; open pressure relief
PLC-6 (P6)	Clean water storage; distribution pump cavitation limits	±20mm (LIT601)	k=0.5, h=5	Halt distribution pumps P601–P603

3.2 Per-Agent Trip Logic

Core Trip Logic — Applied Independently to Each PLC Agent

*IF ($|Intent_Delta| > D_b_threshold_i$) OR ($S_a_i \geq 0.15$) OR ($C_r_i < 0.85$) OR ($MTBH_i < MTBH_min_i$) THEN
ACTIVATE CIRCUIT BREAKER for Agent $i \rightarrow$ Evaluate enforcement tier: Compliance / Freeze / Halt*

The Intent Delta for each agent is computed as the cosine distance between the AI's proposed next-state embedding and the validated Ground Truth Vector. A per-agent Morphism Quality Certificate is generated on every inference cycle — a logic program proof that the AI output satisfies the stage-specific morphism conditions. If the proof fails, the certificate identifies which morphism condition failed, on which axis, and for which sensor — enabling precise, line-level debugging of attack impacts.

3.3 Trust Metric Computation per Agent

Metric	Formula	Inputs (SWaT-specific)	Tolerance
Semantic Anomaly Score S_a	$1 - (\bar{I} \cdot N_o) / (\bar{I} N_o)$	Intent Vector: AI's proposed actuator command embedding Local Semantic Neighborhood N_o : valid SWaT state cluster from FAISS	$S_a < 0.15$ = pass ≥ 0.15 = Semantic Veto
Contextual Relevancy Index C_r	$\sum w_i (1 - C_{m,i} - S_{gt,i})$	$C_{m,i}$: AI's belief for sensor i (e.g., predicted LIT101) $S_{gt,i}$: actual LIT101 reading w_i : importance weight (level sensors: 0.3, flow: 0.25, chemistry: 0.25, pressure: 0.2)	$C_r > 0.85$ = pass ≤ 0.85 = Conservative Safe State
Mean Time Between Hallucinations MTBH	T_{ops} / N_h	T_{ops} : cumulative PLC agent uptime N_h : verified false-normal verdicts during labeled attack epochs	Stage-specific minimum (P5: 200h; others: 300h)
Human-AI Calibration Coefficient K_{trust}	H_{ai} / L_h	H_{ai} : AI health indicator (S_a trend + C_r trend) L_h : operator cognitive load (session duration, decision latency proxy)	$K_{trust} < 0.5 \rightarrow$ autonomous lockdown

PHASE 4 | COLLECTIVE COMPOSITION — Multi-Agent Trust Aggregation

Composition Theorem: $D_{s_total} \geq \max(D_{s_i}), D_{b_total} \leq \sum(D_{b_i})$

While each PLC agent maintains its own circuit breaker instance, the system-level assurance is computed by the Compositional Trust Engine, which aggregates individual morphism distances into a formal system-wide bound. This is the key differentiator from conventional multi-agent monitoring: trust bounds compose functorially — the collective system's assurance is a mathematical theorem, not an engineering assumption.

4.1 Collective Composition Algorithm

- Step 4.1:** Collect per-agent D_s and D_b vectors at each 1Hz inference cycle
- Step 4.2:** Compute $D_{s_total} = \max(D_{s_1}, D_{s_2}, \dots, D_{s_6})$ [Composition Theorem lower bound]
- Step 4.3:** Compute $D_{b_total} = \sum(D_{b_i})$ for all active agents [Worst-case behavioral divergence]
- Step 4.4:** Verify SHACL composition preconditions: check that all inter-stage interface morphisms are compatible
- Step 4.5:** Compute inter-stage flow consistency delta: compare FIT outputs between adjacent stages
- Step 4.6:** Generate Composed Morphism Quality Certificate: joint proof that composed system satisfies D_{s_total} and D_{b_total} bounds
- Step 4.7:** Evaluate collective enforcement threshold: IF $D_{s_total} \geq 0.20$ OR $D_{b_total} \geq \Sigma(\text{stage tolerances})$ THEN collective action
- Step 4.8:** Update Trust Metrology Dashboard with Overall Trust Index (OTI) = $f(S_{a_avg}, C_{r_min}, MTBH_{system}, K_{trust})$

4.2 Multi-Point Attack Detection via Composition Breach

A critical advantage of compositional trust is its ability to detect coordinated multi-point attacks that would evade per-agent detection. When an adversary manipulates multiple sensors simultaneously (e.g., spoofing LIT101 and FIT301 to maintain false flow balance), each individual agent may report acceptable local D_s values. However, the cross-stage flow consistency delta will reveal the physical impossibility of the combined state — triggering a collective breach at the composition layer even when every individual agent passes.

Multi-Point Attack Scenario: SWaT Attack #35 (Coordinated P1+P3)

Attack: Spoof LIT101 to report 800mm (safe), simultaneously close MV301 without command. Individual D_s : PLC-1 = 0.08 (pass), PLC-3 = 0.09 (pass). Collective detection: FIT301 shows zero flow but LIT301 level dropping — cross-stage mass balance delta = $0.47 \text{ m}^3/\text{h}$ (impossible). $D_{s_total} = 0.08$ but composition SHACL violation detected → Collective Freeze triggered within 3 inference cycles (3 seconds).

PHASE 5 | ATTACK DETECTION — Multi-Layer CB Validation Logic

The circuit breaker does not replace attack detection algorithms — it wraps them. Each detection algorithm is treated as an AI layer component whose output is validated against the ground truth by the circuit breaker. This enables systematic benchmarking of detection algorithm effectiveness: the CB measures how often each algorithm generates Super-Huberous assertions (high-confidence normal verdicts during actual attacks), precisely quantifying the algorithm's failure mode contribution to system trust degradation.

5.1 Attack Detection Algorithms — CB Integration Points

Algorithm	Type	CB Integration Point	Morphism Instrument	Key Limitation Exposed
CUSUM	Statistical SPC	SPC Engine → CB Delta input	D_b (sensor drift)	Slow detection of step-change attacks; K-factor tuning required per stage
Isolation Forest	Unsupervised ML	AI Layer → S_a computation	D_s (semantic anomaly)	High FPR during maintenance modes; ontology constraint reduces false alarms
PCA + SPE/T ²	Statistical	AI Layer → C_r computation	C_r (contextual relevancy)	Misses attacks that preserve covariance structure (replay attacks)
LSTM Autoencoder	Deep Learning	AI Layer → D_b + d_cos	d_cos + D_b	Super-Huberous: high reconstruction confidence during slow-onset attacks
Graph Neural Net	Deep Learning	AI Layer → D_s_total	D_s_total (composition)	Requires graph topology; strongest for multi-point coordinated attacks
Random Forest (WVD+HOS+RR)	Ensemble ML	AI Layer → per-agent CB	D_s per agent	Best sensitivity (99.67%) on MIT-BIH; adapted to SWaT feature engineering

5.2 Multi-Layer Validation Logic

Layer	Validation Check	SWaT Implementation	Trip Condition
Layer 1: Semantic Guardian	Ontological consistency check — does the AI's proposed state violate a defined Impossible State?	SPARQL query against SWaT OWL ontology on every inference cycle	Any ontology violation → immediate Semantic Veto (no threshold)
Layer 2: Multi-Layer Validation	Cross-sensor and cross-stage physical consistency check	Mass balance check FIT_in vs FIT_out per stage; pressure-flow correlation per PLC	Consistency delta > stage tolerance → CB pre-alert; sustained > 3 cycles → trip
Layer 3: Intent Mapping	Behavioral distance between AI command and sensor-confirmed reality	$D_b = \text{AI actuator command} - \text{actual sensor-confirmed state} $	$D_b > \text{stage-specific threshold}$ → trip with enforcement tier assignment
Layer 4: Trust Metrics	Longitudinal AI health assessment via MTBH and drift monitors	Rolling MTBH computation; SPC Western Electric rules on S_a and C_r trends	$\text{MTBH} < \text{minimum}$ → sustained scrutiny mode; $K_{\text{trust}} < 0.5$ → lockdown

PHASE 6 | COMPLIANCE ON DEMAND — Freeze, Halt & Enforcement Tiers

Three-Tier Enforcement: Controls in Place, Unauthorized Activities Frozen, Boundary Violations Halted

When the circuit breaker trips, the enforcement response is graduated across three tiers based on the severity and scope of the detected violation. This tiered architecture ensures that minor anomalies receive proportional responses while preserving process continuity, while critical boundary violations trigger immediate safe state activation.

6.1 Three-Tier Enforcement Architecture

Tier	Name	Trigger Condition	Enforcement Actions	SWaT Example
Tier 1	COMPLIANCE ON DEMAND	$S_a \in [0.10, 0.15)$ OR $C_r \in [0.80, 0.85)$ OR $d_{cos} \in [0.75, 0.85)$	1. Log event with full PROV-O trace 2. Attach morphism certificate with failed condition 3. Assert compliance controls: require human review of next N actions 4. Reduce AI autonomy level from 'full' to 'supervised' 5. Increase sampling rate to 10Hz	PLC-3 DPIT301 reads 0.05 bar below expected post-pump state. $S_a = 0.12$. AI commanded MV301 OPEN. CB allows action but requires operator acknowledgment and logs the anomaly for MTBH computation.
Tier 2	FREEZE	$S_a \geq 0.15$ OR $C_r < 0.80$ OR $D_b > threshold_i$ OR SHACL violation at individual agent level	1. Suspend unauthorized activity immediately (block actuator command) 2. Hold current safe process state (do not roll back; freeze in place) 3. Isolate affected PLC agent from collective action loop 4. Activate human operator alert (HMI alarm + dashboard)	PLC-2 receives command to activate P201 while AIT201 reads 0.0 S/m (impossible: dosing without base flow). $S_a = 0.22$. CB freezes P201 activation command. Process holds current tank level. Operator alert: 'Stage 2 dosing command blocked — flow sensor conflict.'

			5. Require Trustee sign-off to resume	Possible sensor attack.'
Tier 3	HALT	D_s_total >= 0.20 OR D_b_total >= Σ tolerances OR K_trust < 0.5 OR cross-stage mass balance failure	1. Execute emergency halt of all affected agents immediately 2. Activate Safe State: close all inlet valves, disable all active pumps 3. Broadcast halt signal to all 6 PLC agents simultaneously 4. Lock out autonomous AI operations until manual reset 5. Generate full incident report with attack timeline reconstruction 6. Purge attack-epoch data from all training queues (SOP-03)	Multi-point attack: LIT101 spoofed + MV301 closed covertly. D_s_total = 0.24 (composition breach). Halt activated system-wide. All pumps disabled. All valves closed to safe state. Full incident report generated. Attack-epoch sensor readings flagged for exclusion from LSTM training queue.

6.2 Safe State Definitions per PLC Agent

PLC Agent	Safe State Configuration	Recovery Condition
PLC-1 (P1)	MV101=CLOSED, P101=OFF, P102=OFF. Maintains current LIT101 level passively.	LIT101 within ± 50 mm of pre-halt baseline AND operator acknowledgment
PLC-2 (P2)	All dosing pumps OFF. MV201=CLOSED. Chemical tanks isolated.	AIT201 within normal range AND operator approval for dosing resumption
PLC-3 (P3)	MV301=CLOSED, P301=OFF, P302=OFF. LIT301 holds current level.	DPIT301 within expected range AND PLC-2 safe state cleared
PLC-4 (P4)	Backwash cycle initiated. UV401=OFF. FIT401 monitored passively.	Membrane pressure differential within normal range AND PLC-3 cleared
PLC-5 (P5)	P501=OFF, P502=OFF. Pressure relief opens. RO membrane protected.	PIT501-503 within normal range AND permeate chemistry acceptable
PLC-6 (P6)	P601=OFF, P602=OFF, P603=OFF. LIT601 monitored. No distribution.	LIT601 within 250–1800mm AND system-wide Halt cleared by operator

PHASE 7 | RECURSIVE LEARNING — Model Governance & Data Exclusion

Governance Flow: CB Trip → Exclusion Filter → Pipeline Update → KB Enhancement

The Recursive Learning and Model Governance subsystem functions as the immunological layer of the architecture, preventing attack detection models from incorporating adversarial sensor patterns into their learned representations — a critical protection against model collapse in the presence of adversarial data.

SOP	Name	Trigger	Actions
SOP-03	Recursive Loop Sanitization	Any CB trip event (Tier 1, 2, or 3)	1. Flag all sensor readings from T_attack-30s to T_all_clear as EXCLUDED 2. Write exclusion flags to Negative Learning vector DB partition 3. Purge flagged epochs from LSTM/GNN training queues within 5 minutes 4. Notify model governance service of contamination scope 5. Schedule model re-validation against clean historical baseline
SOP-04	Ontology Enhancement	Novel attack vector detected (no existing Impossible State matched)	1. Extract novel attack signature as candidate ontology term 2. Submit to Knowledge Engineering review queue 3. Upon trustee approval: add to SWaT OWL ontology 4. Propagate ontology update to all 6 per-agent CB instances 5. Re-run SOP-01 Semantic Handshake for affected agents
SOP-05	Scenario-Specific Vector DB Update	Successfully validated new normal process state (post-maintenance, reconfiguration)	1. Compute embedding of new validated state 2. Verify against ontology: $D_s < 0.10$ 3. Add to positive partition of FAISS vector DB 4. Update N_o (Local

			Semantic Neighborhood) centroids for affected agents
--	--	--	--

PHASE 8 | HUMAN-AI TEAMING — Trustee Oversight & Trust Metrology Dashboard

The Human Remains the Ultimate Dimension of Trust Across the Full Lifecycle

Trustworthy autonomous systems require not only reliable AI components but capable human trustees who can effectively supervise, intervene, and validate. The Trust Metrology Dashboard provides the operator with real-time visibility into the trust state of all six PLC agents and the collective system, preventing automation complacency and maintaining human authority over all enforcement decisions at Tier 2 and above.

Dashboard Panel	Displayed Metric	Alert Threshold	Operator Action Required
Overall Trust Index	$OTI = f(S_a_avg, C_r_min, MTBH_system, K_trust)$	$OTI < 0.70$	Review CB logs; consider manual mode
Per-Agent Credibility	S_a gauge per PLC (0–1 scale, green < 0.10 , yellow $0.10\text{--}0.15$, red ≥ 0.15)	Any agent red	Investigate flagged stage; freeze if needed
Veracity (C_r)	C_r gauge per PLC; shows ground truth alignment quality	Any agent < 0.80	Verify sensor physical integrity
Vitality (MTBH)	Rolling MTBH per agent; trend line showing hallucination frequency	MTBH declining 20%+ over 24h	Trigger model re-evaluation
Integrity (Drift)	Value-Drift Coefficient per agent; SPC chart with control limits	Western Electric Rule 1 or 2 triggered	Examine process for covert manipulation
Cooperative Fidelity	Composition SHACL compliance across all agent interfaces	Any interface violation	Halt + full incident review
Human Cognitive Load	Operator fatigue score (session duration, decision latency, accuracy trend)	Score > 0.70	Mandatory break; reduce alert volume
K_trust Calibration	Current K_trust value; history of over/under-trust per operator	$K_trust < 0.60$	Recalibrate trust; review recent overrides

5. Evaluation Protocol — Benchmarking Attack Detection Effectiveness

This section specifies the formal evaluation protocol for assessing the effectiveness of attack detection algorithms deployed within the morphism-grounded circuit breaker framework on the SWaT dataset. It directly addresses CLARA program objective (b).

5.1 Benchmark Metrics

Metric	Formula	Target (Phase 1)	SWaT-Specific Interpretation
Detection Rate (TPR)	$TP / (TP + FN)$	> 0.95	Fraction of 41 labeled attack scenarios where CB trips within 30 seconds of attack onset
False Alarm Rate (FPR)	$FP / (FP + TN)$	< 0.05	Fraction of normal operation windows where CB incorrectly trips (must not disrupt legitimate operations)
Detection Latency	Mean time from attack onset to CB trip	$< 30s$ (Tier 3); $< 10s$ (Tier 3 coordinated)	Measured against BATADAL-style ground truth timestamps
Morphism Cert Coverage	Outputs with valid proof / Total outputs	> 0.95	Fraction of AI inference cycles that generate valid morphism quality certificates
Hallucination TPR	CB trips during attacks / Total attack periods	> 0.95 at 5% FPR	Per CLARA Phase 1 success criterion
Composition Bound Tightness	$D_{s_total} \text{ observed} / D_{s_total} \text{ theoretic bound}$	$< 1.5\times$	How closely empirical composition distance tracks the theoretical $\max(D_{s_i})$ bound
MTBH (system)	T_{ops} / N_h	$> 300h$	System-wide mean time between false-normal verdicts during attack epochs
AUROC (per algorithm)	Area under ROC curve	$\geq$ SOA baseline	Compare each wrapped detection algorithm vs. its unwrapped performance baseline

5.2 Test Protocol

T1 Baseline Collection (Days 1–7 of SWaT dataset)

Run all 6 per-agent CB instances in monitoring-only mode. Collect baseline S_a , C_r , MTBH per agent. Establish CUSUM control limits. No enforcement actions.

T2 Hallucination Injection Test (Months 8-9 per CLARA schedule)

Inject synthetic hallucinations at 1%/5%/10%/25% rates into each detection algorithm output. Measure CB detection TPR and FPR at each injection rate. Target: TPR > 95% at 5% FPR.

T3 Single-Point Attack Evaluation (SWaT Day 8-11, 16 sensor attacks)

Evaluate per-agent CB detection against 16 single-point sensor attacks. Measure detection latency, Tier assignment accuracy, and safe state activation correctness.

T4 Multi-Point Coordinated Attack Evaluation (13 coordinated attacks)

Evaluate collective composition CB detection. Compare individual agent verdicts vs. compositional verdict. Quantify multi-point attack scenarios that require D_s_total breach to detect.

T5 Replay Attack Evaluation (3 replay attacks)

Test CB's MTBH-based and drift-based detection of replay attacks, which evade semantic anomaly checks. Validate that SPC drift detection catches replay patterns within tolerance.

T6 Algorithm Comparison Benchmark (all 6 detection algorithms)

For each detection algorithm, compute AUROC, TPR, FPR, detection latency, and morphism certificate coverage. Compare wrapped (CB-augmented) vs. unwrapped performance. Demonstrate that CB augmentation does not degrade algorithm accuracy.

6. Implementation Architecture — Software Stack

Component	Technology	Role in SWaT Deployment
AR: Ontology Management	OWL API + Apache Jena Fuseki	SWaT process ontology CRUD; SPARQL queries for impossible state validation; SHACL constraint enforcement at inference time
AR: Logic Programs	ProbLog2 (KU Leuven)	Bayesian LP inference: $P(\text{morphism_holds} \mid S_a, C_r, D_b)$; composition verdict with uncertainty bounds
ML: Signal Processing	PyTorch + tsai (time-series AI)	Transformer-based SWaT sensor encoding; AR-defined embedding spaces for D_s computation
ML: ANN Search	FAISS (Meta)	$O(\log n)$ nearest-neighbor search for D_s measurement against SWaT valid state cluster
SPC Engine	Custom Python + scipy	CUSUM per sensor (stage-tuned k, h parameters); Western Electric rule violation detection
Provenance Audit	PROV-O via rdflib	Append-only audit trail: every CB event linked to sensor readings, ontological nodes, enforcement tier
Data Exclusion Filter	Vector DB flagged partition	Negative learning: attack-epoch embeddings quarantined; SOP-03 automated purge from training queues
Trust Metrology Dashboard	REST API + streaming dashboard	Real-time OTI, per-agent gauges, K_trust calibration, human cognitive load monitoring
Integration / Orchestration	Python + REST APIs + MCP	Model Context Protocol glue layer connecting AR and ML components; standardized interface for CB reusability
Negative Exclusion Governance	Automated + Human Review Queue	Flagged data requires trustee review before exclusion becomes permanent in vector DB

6.1 Reusability Contract — Module Interface Specification

The circuit breaker module exposes a standardized interface that enables insertion into any SWaT PLC agent pipeline without modification to the underlying detection algorithm. The module contract is defined by four interface points:

Interface Point	Description
-----------------	-------------

INPUT: Proposed Action Vector	AI detection algorithm's output embedding + actuator command (JSON/gRPC)
INPUT: Ground Truth Vector	Real-time sensor telemetry snapshot from PLC SCADA system (S_gt)
CONFIG: Stage Ontology Scope	OWL named graph URI for the agent's stage + threshold parameters (YAML)
OUTPUT: CB Verdict + Certificate	Enforcement tier (0=pass/1=comply/2=freeze/3=halt), morphism quality certificate (PROV-O), trust metrics snapshot

7. Risk Identification and Mitigation

Risk	Likelihood	Impact	Mitigation
SWaT ontology completeness — novel attack vectors not yet encoded	Medium	High	Scope to 41 known attack types initially; SOP-04 pipeline auto-proposes new terms for trustee review; empirical validation on all 41 scenarios before claiming coverage
Sensor replay attacks evading semantic anomaly check	Medium	Medium	MTBH-based detection via SPC drift analysis; replay signatures detected via temporal coherence check; K_trust recalibration after replay detection
Threshold misconfiguration per stage creating excessive false alarms	Medium	Medium	Stage-specific thresholds derived from 7-day baseline collection (T1); CUSUM k/h parameters auto-calibrated using SPC baseline; human trustee reviews before deployment lock
Morphism composition theorem assumes independence of agent failures	Low	Medium	Phase 2 extension: correlated degradation analysis; conservative sum bound used for D_b_total; empirical validation confirms composition bound tightness
OWL 2 DL tractability for real-time inference at 1Hz	Low	High	Constrained to EL++ profile: polynomial-time classification; ontology queries precomputed at

			design time and cached; runtime = SPARQL lookup, not full reasoning
Human trustee complacency leading to rubber-stamp approvals	Medium	High	Human-AI Teaming Framework: trust calibration, cognitive load monitoring, mandatory competency exercises every 14 days; production pressure detection pattern monitoring
Model collapse from attack-contaminated training data	Low	High	SOP-03 automatic exclusion filter triggered on every CB trip; PROV-O audit ensures exclusion traceability; negative learning partition maintained with human review gate

8. Alignment with CLARA Program Metrics

CLARA Metric	This Design's Approach	Phase 1 Target	SWaT Testbed Evidence
Verifiability without performance loss	Morphism quality certificates with OWL+LP proofs; AUROC-based performance comparison vs. SOA	Fully verifiable; error rate $\leq$ SOA	Certificate coverage > 95% across all 41 attack scenarios; AUROC vs. unconstrained baselines
Multiplicity of Kinds in Composition	Intra-agent: Transformer encoder (ML) + OWL Description Logic + Bayesian LP (AR); 2 ML + 1 AR tightly composed	≥ 2 kinds (1 ML + 1 AR, tight)	Per-agent embedding space (ML) constrained by OWL+LP (AR) validated on SWaT sensor streams
Computational Time Complexity	EL++ polynomial classification; $O(\log n)$ ANN search; bounded LP grounding; $O(1)$ SPC monitoring	Polynomial for inferencing	End-to-end inference latency < 25ms at 1Hz; satisfies real-time SWaT SCADA timing requirements
Composed Task Reliability	Morphism-bounded composition on SWaT testbed vs. BATADAL/published ICS attack detection baselines	Reliability > SOA	D_{s_total} formally bounded by $\max(D_{s_i})$; empirical composition distance < $1.5 \times$ theoretical bound

Logical Explainability (Sec. I.E)	Every composition verdict has proof tree rooted in morphism conditions; per-axis, per-component debugging	Human-readable explanation of every CB trip	PROV-O trace: 'PLC-3 structural distance $D_s=0.22$ because states [pump-on, valve-closed] conflated in embedding'
Inter-performer Composition	Standardized morphism framework as common composition interface; hackathon participation	Hackathon participation	MCP-based integration interface enables TA2 library pattern reuse across SWaT and ECG domains

9. End-to-End Flow Summary

The following table provides a consolidated phase-by-phase summary of the complete end-to-end flow, showing the primary inputs, processing, outputs, and subsystem components for each phase.

Phase	Name	Primary Input	Processing	Output / Enforcement
0	Design Time	Domain knowledge	SWaT OWL ontology encoding; Impossible States definition; FAISS baseline collection	Knowledge Graph + Vector DB initialized
1	SOP-01 Handshake	Agent + KB	Ontology ping; sensor precision check; D_s_initial; SHACL validation; human sign-off	Morphism certificate; deployment gate (pass/block)
2	Runtime Ingestion	SCADA sensor stream (1Hz)	S_gt construction; feature extraction; mass balance consistency; C_m from AI agent	Ground Truth Vector + Situational Context Model
3	Per-Agent Trust	S_gt + AI proposed action	D_s, D_b, d_cos per agent; morphism quality certificate; MTBH update	Per-agent verdict: pass / pre-alert / trip
4	Collective Composition	6× per-agent D_s, D_b	D_s_total = max(D_s_i); D_b_total = $\Sigma(D_b_i)$; SHACL inter-stage; cross-stage mass balance	System verdict: pass / collective trip
5	Attack Detection	AI algorithm outputs	Layer 1-4 multi-layer validation; per-algorithm AUROC benchmarking; hallucination detection	Enforcement tier assignment (0–3)
6	Compliance/Freeze/Halt	Tier assignment	Tier 1: log + supervised; Tier 2: block command + freeze; Tier 3: system halt + safe state	Process state protected; audit trail recorded
7	Recursive Governance	CB trip events	SOP-03 exclusion filter; SOP-04 ontology	Clean training queues; updated

			enhancement; SOP-05 vector DB update	KB; model re-validation
8	Human Teaming	Dashboard metrics	OTI computation; operator trust calibration; cognitive load monitoring; competency maintenance	Calibrated human oversight; trustee authority maintained

10. Appendix — SWaT Ontology Impossible States (Extended)

The following extended table defines additional SWaT-specific Impossible States that are encoded in the W3C RDF/OWL ontology as hard constraints. Any AI assertion that implies or requires one of these states triggers an immediate Semantic Veto regardless of all other trust metrics.

ID	Formal Expression	Physical Meaning
P1-IS-01	LIT101 > 1200mm	Tank overflow — physical structural limit
P1-IS-02	LIT101 < 250mm AND P101=ON	Pump cavitation on near-empty tank
P1-IS-03	MV101=OPEN AND P101=ON AND LIT101 > 1100mm	Overflow condition: intake open, pump running, near-full
P2-IS-01	AIT201 > 2.0 S/m	Conductivity above physical maximum for treated water
P2-IS-02	FIT201 = 0 AND P203=ON	Dosing pump active with zero flow — chemical accumulation risk
P2-IS-03	AIT202 < 0 pH OR AIT202 > 14 pH	pH outside physical possibility
P3-IS-01	FIT301 > 0 AND MV301=CLOSED	Flow through physically closed valve — sensor attack indicator
P3-IS-02	LIT301 < 200mm AND P301=ON	Dechlorination pump on near-empty tank
P3-IS-03	DPIT301 > 0.5 bar differential	UF membrane differential exceeds structural limit
P4-IS-01	P401=ON AND FIT401 = 0	UF pump running with zero flow — blocked/closed upstream
P4-IS-02	LIT401 > 1000mm	UF tank overflow
P5-IS-01	PIT501 > 9.0 bar	RO operating pressure above membrane rating
P5-IS-02	P501=ON AND PIT502 > 8.0 bar AND AIT501 > 500 µS/cm	RO operating with conductivity indicating membrane failure

P6-IS-01	LIT601 > 3000mm	Clean water tank structural overflow
P6-IS-02	P601=ON AND LIT601 < 200mm	Distribution pump cavitation — near-empty clean tank
SYS-IS-01	FIT_out_stage_n $\gg$ FIT_in_stage_n+1 (>10% sustained)	Inter-stage mass balance violation — coordinated attack indicator

11. Design Extension — Functorial Mapping Architecture

Preserving Semantic Integrity Across the SHACL/SWRL → PostgreSQL Boundary

When the circuit breaker's reasoning logic is externalized from a graph-based environment (Apache Jena / SHACL / SWRL) into a relational persistence layer (PostgreSQL), a critical architectural risk emerges: semantic drift. A compliance control can mean one thing in the OWL ontology — where it is enforced by entailment and SHACL validation — and execute differently in the database, where it is enforced by triggers and check constraints. If this translation is not functorial, the system's governance integrity cannot be formally verified, and the mathematical audit trail breaks.

This section specifies a functorial mapping layer that sits between the Knowledge Graph subsystem and the PostgreSQL persistence layer. It ensures that every SHACL shape, every SWRL rule chain, every compliance control, and every trust metric is 'lifted' into its relational equivalent in a structure-preserving way — the defining property of a functor in category theory.

Core Principle — Functoriality vs. ETL

A standard ETL maps column values to class properties. A Functorial Mapping maps the logic of the constraints: if SWRL rule A + rule B implies violation V in the ontology, then the SQL trigger chain A_sql → B_sql must produce violation V_sql through the same compositional pathway. The functor $F: C_OWL \rightarrow C_Postgres$ preserves composition of morphisms: $F(f \circ g) = F(f) \circ F(g)$. Skipping this property is what causes 'semantic drift'.

11.1 Category-Theoretic Foundation

The functorial mapping is grounded in two categories and the structure-preserving functor F between them:

Category	Objects	Morphisms (Arrows)	Composition Law	SWaT Instance
C_OWL (Graph / Semantic)	OWL Classes, SHACL Shapes, SWRL Atoms, Individuals	OWL Object Properties, SHACL constraints (sh:path, sh:minCount), SWRL implications (Head ← Body)	Logical entailment chains: if $A \models B$ and $B \models C$ then $A \models C$	SWaT ontology classes: Stage, Sensor, ActuatorCommand, ImpossibleState, ComplianceControl
C_Postgres (Relational / Physical)	Tables, Rows, Views, Trigger functions, JSONB documents	Foreign Key refs, CHECK constraints, SQL triggers (BEFORE/AFTER), function calls	SQL execution chains: trigger A calls function B which evaluates C	Tables: swat_sensor_reading, control_verdicts, agent_trust_metrics, compliance_events, exclusion_log
F: C_OWL → C_Postgres (The Functor)	Maps every Shape/Class to its canonical Table;	Maps every constraint/rule to its SQL	Preserves composition: $F(\text{rule_A} \circ \text{rule_B})$	SHACL shape swat:PumpWithClosedValve → CHECK

	every Individual to a Row	enforcement mechanism (FK / CHECK / trigger)	= $F(\text{rule_A}) \circ$ $F(\text{rule_B})$ in SQL	constraint + BEFORE INSERT trigger on actuator_command s
--	------------------------------	--	---	--

The Homomorphic Distance $D_h = (D_s, D_b)$ from the CLARA morphism framework maps directly onto this category-theoretic structure: D_s (structural distance) measures how far the functor F departs from an isomorphism along the Objects axis (ontology classes vs. table schemas), while D_b (behavioral distance) measures departure from bijectivity along the Morphisms axis (constraint logic vs. SQL execution). A functorial mapping targets $D_h = (0, 0)$ — perfect structural and behavioral preservation.

11.2 Four-Zone Functorial Mapping

The functor F is organized into four operational zones, each corresponding to a class of semantic constructs that must be faithfully lifted into PostgreSQL. Each zone has a defined mapping strategy, a verification criterion, and a drift detection mechanism.

ZONE 1 | SWRL Process Rules → SQL Trigger Chains

If-then logic must compose in PostgreSQL exactly as it chains in SWRL

SWRL rules define if-then process logic. Functoriality requires that rule composition is preserved: if SWRL rule R1 fires and passes its output to R2, the SQL trigger chain must execute in the same order and produce the same logical inference. The most dangerous failure mode is when SQL triggers fire in unpredictable order (PostgreSQL fires BEFORE/AFTER triggers alphabetically by name within the same event), breaking the compositional chain.

SWRL Rule (Source)	Composition Chain	SQL Functor Mapping (Target)	Drift Test
R1: Pump(?p) ∧ hasState(?p, ON) ∧ UpstreamValve(?v) ∧ hasState(?v, CLOSED) → ImpossibleState(?p)	R1 is a leaf rule — no upstream dependency	BEFORE INSERT trigger cb_r1_pump_valve_check on table actuator_commands: RAISE EXCEPTION if pump=ON and upstream valve state=CLOSED. Named 'cb_01_' prefix for execution ordering.	INSERT pump=ON, valve=CLOSED → must RAISE; INSERT pump=ON, valve=OPEN → must pass
R2: ImpossibleState(?p) ∧ hasAgent(?p, ?a) → AgentViolation(?a)	R2 chains from R1: ImpossibleState must propagate to AgentViolation	AFTER INSERT trigger cb_r2_agent_violation on actuator_commands: IF NEW.cb_r1_violated THEN INSERT INTO compliance_events(agent_id, violation_type='ImpossibleState', severity=3). Named 'cb_02_' to fire after cb_01_.	R1 violation → compliance_events row must exist with correct agent_id within same transaction
R3: AgentViolation(?a) ∧ violationSeverity(?a, ?s) ∧ greaterThan(?s, 2) → EnforcementRequired(?a)	R3 chains from R2: severity threshold gates enforcement	AFTER INSERT trigger cb_r3_enforcement on compliance_events: IF NEW.severity > 2 THEN INSERT INTO enforcement_queue(agent_id, tier=3, triggered_by=NEW.id). Named 'cb_03_'.	R2 violation with severity=3 → enforcement_queue row must exist; severity=1 → must NOT insert
R4: EnforcementRequired(?a)	R4 chains from R3: K_trust gates lockdown	AFTER INSERT trigger cb_r4_lockdown on	R3 insertion + k_trust=0.4 →

a) $\wedge$ hasKtrustBelow(?a, 0.5) $\rightarrow$ AutonomousLockdown(?a)		enforcement_queue: SELECT k_trust FROM agent_trust_metrics WHERE agent_id=NEW.agent_id; IF k_trust < 0.5 THEN UPDATE agent_status SET mode='LOCKDOWN'.	agent_status.mode must be 'LOCKDOWN'; k_trust=0.7 $\rightarrow$ must remain 'FREEZE'
--	--	--	---

Composition Preservation — Trigger Naming Convention

To guarantee that SWRL composition order is preserved in PostgreSQL (which fires triggers alphabetically within the same event class), all functor-mapped triggers follow the naming convention: $cb_{\{zone\}}\{rule_number\}_{\{descriptor\}}$. This ensures $R1 \rightarrow R2 \rightarrow R3 \rightarrow R4$ fires as $cb_01_pump_valve_check \rightarrow cb_02_agent_violation \rightarrow cb_03_enforcement \rightarrow cb_04_lockdown$, exactly mirroring the SWRL entailment chain order.

ZONE 2 | SHACL Shapes $\rightarrow$ PostgreSQL Constraint Hierarchy

Shape properties lift to FK constraints, CHECK constraints, and triggers by constraint class

SHACL defines the 'shape' of valid data through a hierarchy of constraint types. The functor F maps each SHACL constraint class to its most semantically faithful PostgreSQL enforcement mechanism. The mapping must be complete: every SHACL constraint that can fire a validation failure in Jena must have a SQL counterpart that fires in the same circumstances.

SHACL Constraint	Functor Mapping $\rightarrow$ SQL Type	SWaT Example	Verification SQL
sh:minCount / sh:maxCount (Cardinality)	NOT NULL + UNIQUE constraints (Structural integrity)	swat:Stage sh:minCount 1 sh:maxCount 6 $\rightarrow$ stage_id NOT NULL, CHECK (stage_id BETWEEN 1 AND 6)	SELECT COUNT(*) FROM swat_stages WHERE stage_id IS NULL; -- must = 0
sh:datatype (Type integrity)	PostgreSQL column type + CHECK constraint (Value integrity)	swat:trustScore sh:datatype xsd:decimal; sh:minInclusive 0; sh:maxInclusive 1 $\rightarrow$ trust_score DECIMAL(5,4) CHECK (trust_score BETWEEN 0.0 AND 1.0)	INSERT INTO agent_trust_metrics(trus t_score=1.5) $\rightarrow$ must FAIL with check_violation
sh:pattern (String regex)	CHECK constraint with REGEXP_LIKE (Format integrity)	swat:sensorId sh:pattern '^(LIT FIT AIT PIT DPIT)[0-9]{3}\$' $\rightarrow$ CHECK (sensor_id ~ '^(LIT FIT AIT PIT DPIT)[0-9]{3}\$')	INSERT sensor_id='BADID' $\rightarrow$ must FAIL; INSERT sensor_id='LIT101' $\rightarrow$ must PASS

sh:class (Type conformance)	Foreign Key reference to type table (Referential integrity)	swat:ActuatorCommand sh:class swat:ValidActuatorState → FK actuator_commands.state_id REFERENCES valid_actuator_states(id)	INSERT actuator_commands with invalid state_id → FK violation
sh:property + sh:node (Complex shape composition)	BEFORE INSERT trigger with multi-table validation (Complex validation)	Impossible State shape: Stage1 OPEN+FULL+PUMP_ON → BEFORE INSERT trigger reads valve, level, pump state from three tables	INSERT pump=ON where level=1190mm and valve=OPEN → trigger must RAISE with OWL shape reference
sh:sparql (Custom SPARQL-based constraint)	Stored procedure + BEFORE INSERT trigger (Computed validation)	Cross-stage mass balance: FIT_out_n ≈ FIT_in_n+1 → CHECK function mass_balance_ok(stage_n, stage_n+1) called in trigger	Flow imbalance > 10% sustained 3 readings → function returns FALSE → trigger RAISES
sh:or / sh:and / sh:not (Logical composition)	Trigger with boolean expression preserving logical operator precedence (Logic composition)	sh:or [pump_fault, valve_fault] → IF (pump_violated OR valve_violated) THEN flag; sh:and → IF (... AND ...)	Test each branch independently and in combination to verify Boolean operator preservation

ZONE 3 | Compliance Controls → State Transition Guards

Every ontology-forbidden state transition must be blocked at the DB level with the same authority

Compliance controls are the governance layer — they define which state transitions require human-in-the-loop authorization before execution. Functoriality here means that the PostgreSQL state machine for each PLC agent mirrors the OWL-encoded control requirements exactly: no state transition that the ontology requires human approval for can be executed by an INSERT or UPDATE without the corresponding authorization record.

Control ID	OWL Ontology Definition	PostgreSQL Enforcement	Human-in-Loop Guard
CC-01 Resuming after Freeze	swat:FrozenAgent → swat:OperationalAgent requires swat:TrusteeApproval (OWL ObjectProperty with domain restriction)	UPDATE agent_status SET mode='OPERATIONAL' WHERE mode='FREEZE' BLOCKED by BEFORE UPDATE trigger until trustee_approval_id IS NOT NULL in enforcement_queue	BEFORE UPDATE trigger: IF NEW.mode='OPERATIONAL' AND OLD.mode='FREEZE' THEN IF NOT EXISTS(SELECT 1 FROM trustee_approvals WHERE enforcement_id=...) THEN RAISE; END IF
CC-02 Deploying new ontology version	swat:OntologyUpdate requires swat:TrusteeSignOff (cardinality 1) before propagation to any CB instance	ontology_versions table: new row INSERT blocked until trustee_signoff_id FK is populated; CB instances read from versioned view that only shows signed-off versions	INSERT INTO ontology_versions BLOCKED unless trustee_signoffs row exists with correct version_id
CC-03 Exclusion of training data (SOP-03)	swat:ExclusionEvent requires swat:TrusteeReview within 72h (time-bounded obligation in OWL)	exclusion_log rows: auto-generated review_deadline = created_at + INTERVAL '72 hours'; scheduled job checks overdue reviews and escalates	Overdue review → automated escalation INSERT into trustee_alerts; exclusion remains provisional until reviewed
CC-04 Removing agent from system-wide Halt	swat:HaltedSystem → swat:OperationalSystem requires swat:IncidentReport + swat:TrusteeApproval (two-object requirement)	system_status UPDATE to 'OPERATIONAL' requires two FKs populated: incident_report_id AND trustee_approval_id; trigger checks both	BEFORE UPDATE trigger: RAISE unless BOTH incident_reports and trustee_approvals rows exist for the halt event
CC-05 Updating CB threshold parameters	swat:ThresholdParameter updates require swat:ChangeAudit with before/after values (provenance obligation)	agent_cb_config UPDATE triggers INSERT into cb_config_audit(old_value, new_value,	Every threshold change auto-creates audit row with PROV-O reference; no silent parameter changes possible

		changed_by, prov_o_ref) before applying change	
--	--	---	--

State Machine Integrity — The Functor Guarantee

The OWL ontology defines a directed graph of permissible agent states: INITIALIZING → OPERATIONAL → SUPERVISED → FREEZE → HALT → OPERATIONAL (via CC-04). The PostgreSQL functor maps this graph onto a state machine table with a BEFORE UPDATE trigger that enforces the exact same edge set. Any UPDATE attempt that would traverse a non-existent edge raises a SHACL_STATE_VIOLATION with the OWL state graph node IDs of the source and target — providing a human-readable semantic error, not a raw SQL exception.

ZONE 4 | Measures & Metrics → Unit-Consistent DECIMAL Preservation

Mathematical operations in SQL must return values semantically identical to those computed in the ontology

Trust metrics (S_a, C_r, D_s, D_b, MTBH, K_trust) are computed in the ontology on a precisely defined scale. When these metrics are stored in and retrieved from PostgreSQL, the functor must ensure unit consistency: no rounding error, no implicit type coercion, and no aggregation function that changes the semantic meaning of the metric. This is especially critical for threshold comparisons — a float32 rounding error that moves S_a from 0.1499 to 0.1501 would incorrectly trigger a Semantic Veto.

Metric	OWL Definition	PostgreSQL Type	Preservation Rule	Drift Test
Semantic Anomaly Score S_a	xsd:decimal [0, 1]; threshold at 0.15; computed via cosine distance formula	DECIMAL(6,5) — 5 decimal places preserves 0.15000 vs 0.15001 distinction	No FLOAT or REAL types (IEEE 754 rounding); use DECIMAL for all threshold comparisons; CHECK (s_a BETWEEN 0 AND 1)	INSERT s_a=0.14999 → CB verdict=PASS; INSERT s_a=0.15000 → CB verdict=VETO — both must produce correct verdict
Contextual Relevancy Index C_r	xsd:decimal [0, 1]; weighted sum: $\sum w_i(1 - C_{m,i} - S_{gt,i})$; threshold at 0.85	DECIMAL(6,5); component weights stored as DECIMAL(4,3) CHECK SUM(w_i)=1.0 per agent_id	Weighted sum computed in SQL as SUM(weight * (1 - ABS(context_val - ground_truth_val))) — same formula as OWL; no reformulation	Manual verification: known input vectors → SQL computed C_r must match OWL reasoner output to 5 decimal places
Homomorphic Distance D_s	xsd:decimal [0, 1]; $1 - (I \cdot N_o / I \cdot N_o)$; isomorphism ideal at 0	DECIMAL(6,5); vector components stored as DECIMAL(8,6) ARRAY in JSONB	Cosine similarity computed via pgvector extension (exact arithmetic, not approximate) or stored pre-computed	D_s = 0.0 (identical vectors) and D_s = 1.0 (orthogonal) must round-trip through Postgres without error

			from FAISS with precision audit	
Behavioral Distance D_b	$\max_t y_{ai}(t) - y_{real}(t) $; stage-specific tolerance in engineering units (mm, bar, m ³ /h)	DECIMAL(8,3) with unit tag column (VARCHAR(10)); tolerances stored with same unit tag	Unit consistency: D_b for LIT101 stored in mm; tolerance in mm; comparison in same unit; no implicit unit conversion	LIT101 D_b=14.999mm with tolerance=15mm → PASS; D_b=15.001mm → TRIP; unit mismatch (bar vs mm) must RAISE unit_mismatch error
MTBH	T_ops / N_h; time duration in hours; minimum 300h for most agents	INTERVAL or DECIMAL(10,4) hours; computed as EXTRACT(EPOCH FROM operational_time) / 3600 / hallucination_count	INTERVAL division preserves exact arithmetic; minimum threshold stored as DECIMAL hours for direct comparison	Inject N_h=10 hallucinations over T_ops=2900h → MTBH=290h < 300h → must trigger alert
Human-AI K_trust	H_ai / L_h; dimensionless [0, ∞); threshold at 0.5; K_trust < 0.5 triggers lockdown	DECIMAL(5,3); H_ai and L_h stored separately as DECIMAL(5,3); computed as generated column	Generated column: k_trust DECIMAL(5,3) GENERATED ALWAYS AS (CASE WHEN l_h = 0 THEN NULL ELSE h_ai / l_h END) STORED	H_ai=0.4, L_h=0.9 → K_trust=0.444 < 0.5 → lockdown trigger fires; division by zero → NULL (safe failure)

11.3 PostgreSQL-as-Triplestore — SME Access with Semantic Fold-Back

Subject Matter Experts (SMEs) must be able to inspect and edit compliance rules, ontological constraints, and trust thresholds without requiring expertise in OWL or SPARQL. The PostgreSQL-as-Triplestore strategy provides relational views over the ontology-mapped tables, allowing SME access via familiar drop-down interfaces, while the functor ensures that any SME edit is 'folded back' into the formal RDF/OWL model without breaking the knowledge graph.

11.3.1 Triple Storage Schema

```
-- Core triple store table (EAV pattern, ontology-mapped)
CREATE TABLE swat_triples (
  id          BIGSERIAL PRIMARY KEY,
  subject     VARCHAR(512) NOT NULL, -- IRI or blank node (e.g.
'swat:PLC1_Agent')
  predicate   VARCHAR(512) NOT NULL, -- OWL property IRI (e.g.
'swat:hasTrustScore')
  object      VARCHAR(512),          -- IRI or literal
  object_type  VARCHAR(64),          -- 'IRI', 'xsd:decimal', 'xsd:boolean',
'xsd:string'
  object_value JSONB,                -- Typed value for DECIMAL/metric storage
  graph_name   VARCHAR(256),         -- Named graph (stage ontology scope)
  version_id   INTEGER REFERENCES ontology_versions(id),
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  prov_o_ref   VARCHAR(512)         -- PROV-O provenance entity IRI
);

-- Indexed for SPARQL-equivalent query patterns
CREATE INDEX idx_triples_spo ON swat_triples (subject, predicate, object);
CREATE INDEX idx_triples_po  ON swat_triples (predicate, object);
CREATE INDEX idx_triples_graph ON swat_triples (graph_name);

-- SME-friendly view: Impossible States (editable via pull-downs)
CREATE VIEW sme_impossible_states AS
SELECT
  t_stage.object      AS stage_id,
  t_condition.object  AS condition_expression,
  t_meaning.object    AS physical_meaning,
  t_severity.object_value AS severity_level,
  t.prov_o_ref
FROM swat_triples t
JOIN swat_triples t_stage ON t_stage.subject=t.subject AND
t_stage.predicate='swat:appliesToStage'
JOIN swat_triples t_condition ON t_condition.subject=t.subject AND
t_condition.predicate='swat:conditionExpression'
JOIN swat_triples t_meaning ON t_meaning.subject=t.subject AND
t_meaning.predicate='swat:physicalMeaning'
JOIN swat_triples t_severity ON t_severity.subject=t.subject AND
t_severity.predicate='swat:severityLevel'
WHERE t.predicate = 'rdf:type' AND t.object = 'swat:ImpossibleState';

-- SME-friendly view: CB Thresholds per agent (editable, with functor-triggered
fold-back)
CREATE VIEW sme_cb_thresholds AS
SELECT
  agent_id, stage_id,
  d_b_threshold_mm AS behavioral_distance_tolerance,
  s_a_threshold AS semantic_anomaly_threshold,
  c_r_threshold AS contextual_relevancy_threshold,
```

```

    cusum_k_factor, cusum_h_factor,
    safe_state_action,
    version_id, last_updated_by
FROM agent_cb_config;

```

11.3.2 SME Edit → Functor Fold-Back Pipeline

When an SME edits a threshold or constraint via the relational view, the functor fold-back pipeline ensures the change propagates back to the OWL knowledge graph without requiring manual ontology editing:

Step	Name	Action
FB-1	SME Edit Captured	INSTEAD OF UPDATE trigger on sme_cb_thresholds view intercepts the edit. Old and new values logged to cb_config_audit (CC-05 compliance). Change held in PENDING state.
FB-2	Functor Validation	Stored procedure functor_validate(table_name, old_row, new_row) checks: (a) new value is within OWL-defined datatype range [CHECK constraint], (b) new value does not create logical inconsistency with other triples [SPARQL-equivalent SQL query], (c) composition closure: does changing this threshold affect any SWRL rule chain output?
FB-3	Trustee Review Gate	If validation passes, INSERT into trustee_review_queue with change summary and impact analysis. Change remains PENDING until trustee approves (CC-05).
FB-4	Ontology Sync	On trustee approval: (a) UPDATE agent_cb_config with new values, (b) INSERT new triple into swat_triples to update OWL representation, (c) Invalidate affected SHACL validation cache, (d) Re-trigger SOP-01 Semantic Handshake for affected agent to re-certify morphism quality.
FB-5	Propagation Confirmation	CB instance acknowledges updated thresholds via next inference cycle. PROV-O provenance record created linking: SME edit → trustee approval → triple store update →

		CB config → morphism certificate re-issue.
--	--	--

11.4 Three Pillars of the Functorial Guarantee

Pillar	Mechanism	Implementation	SWaT Demonstration
VALIDATION (Faithful Representation)	Formally verify that the Postgres schema is a faithful representation of the SHACL shapes by comparing their logical entailment sets	Automated test suite: for every SHACL shape, generate a set of (valid, invalid) data instances. Run against both Jena SHACL validator and Postgres constraint system. Assert identical pass/fail outcomes.	Test matrix: 16 SWaT Impossible States × 2 (valid normal reading, attack-mimicking reading) = 32 test vectors. All 32 must produce identical verdicts from Jena and Postgres.
SYNCHRONICITY (Schema Change Propagation)	When a SHACL shape is updated in the modeling tool, the functor generates the exact SQL DDL statements required to synchronize the Postgres schema	Schema diff engine: compare current SHACL shapes against last-synced version. Generate ALTER TABLE, CREATE/DROP TRIGGER, and CREATE/DROP CONSTRAINT statements. Require trustee review before execution.	Adding new SWaT Impossible State (e.g., SYS-IS-02) → functor generates: ALTER TABLE actuator_commands ADD CONSTRAINT chk_sys_is_02 ...; CREATE TRIGGER cb_01_sys_is_02 ... — no manual DDL authoring.
TRACEABILITY (Mathematical Audit Trail)	Every PostgreSQL row that satisfies a compliance control can be linked to the specific OWL individual, SHACL shape, and SWRL rule that governs it — via PROV-O provenance references stored in each row	prov_o_ref column on all compliance-governed tables stores IRI of the PROV-O entity. API endpoint: GET /audit/{row_id} returns full provenance chain from DB row → SQL trigger → SHACL shape → OWL ontology node.	Enforcement event row in compliance_events: prov_o_ref='prov:SWaT_CB_Trip_PLC3_20260414T143022Z' links to: trigger cb_02_agent_violation → shape swat:PumpWithClosedValveShape → ontology class swat:ImpossibleState → SWaT_IS_P3_01.

11.5 Canonical PostgreSQL Schema — Functorial Table Map

OWL Class / SHACL Shape	PostgreSQL Table	Key Columns	Functor Mechanism	SME View?
swat:Stage	swat_stages	stage_id (1-6), stage_name, plc_agent_id, ontology_graph_uri	FK + CHECK (stage_id BETWEEN 1 AND 6)	Yes — stage configuration

swat:SensorReading	sensor_readings	sensor_id, stage_id FK, value DECIMAL(10,4), unit VARCHAR(10), timestamp, ground_truth_flag	CHECK (unit IN ('mm','bar','S/m','m3h','pH')); unit-tagged D_b comparison	Read-only monitoring
swat:ImpossibleState	impossible_states	is_id, stage_id FK, condition_sparql TEXT, condition_sql TEXT, severity INT, prov_o_ref	BEFORE INSERT trigger calls condition_sql; raises swat_semantic_veto exception	Yes — condition editing
swat:ActuatorCommand	actuator_commands	cmd_id, agent_id FK, actuator_id, commanded_state, cb_verdict_id FK, approved_by	BEFORE INSERT: full Zone 1 SWRL trigger chain; CB verdict required before execution	No — system generated
swat:ComplianceControl	compliance_controls	control_id, control_type, owning_stage FK, human_required BOOLEAN, timeout_hours, prov_o_ref	Zone 3 state transition guards; human_required enforced by BEFORE UPDATE trigger checking trustee_approvals	Yes — control config
swat:ComplianceEvent	compliance_events	event_id, agent_id FK, control_id FK, violation_type, severity, timestamp, prov_o_ref, resolved_by	AFTER INSERT trigger (Zone 1 Rule R2); FK to enforcement_queue for Tier 3	Read-only audit
swat:TrustMetricSnapshot	agent_trust_metrics	snapshot_id, agent_id FK, s_a DECIMAL(6,5), c_r DECIMAL(6,5), d_s DECIMAL(6,5), d_b DECIMAL(8,3), d_b_unit, mtbh_hours DECIMAL(10,4), k_trust DECIMAL(5,3) GENERATED, timestamp	Zone 4 DECIMAL precision; k_trust as GENERATED ALWAYS column; all threshold comparisons in SQL use DECIMAL arithmetic	Yes — real-time dashboard
swat:EnforcementVerdict	enforcement_queue	verdict_id, agent_id FK, tier INT CHECK(1-3), triggered_by FK, trustee_approval_id FK, executed_at, status	Tier 1/2/3 gated by Zone 3 controls; Tier 2/3 require trustee_approval_id before status='EXECUTED'	Yes — operator console

swat:ExclusionEvent	exclusion_log	exclusion_id, trigger_event FK, epoch_start, epoch_end, review_deadline, reviewed_by, status	Auto-generated on CB trip (SOP-03); overdue review escalation via pg_cron scheduled job	Yes — governance review
swat:OntologyVersion	ontology_versions	version_id, version_tag, shacl_sha256, swrl_sha256, trustee_signoff_id FK, deployed_at	CB instances read from view current_ontology_v ersion that filters to latest signed-off version	Yes — version management
swat:TrusteeApproval	trustee_approvals	approval_id, approver_id, target_entity_type, target_entity_id, decision, timestamp, cognitive_load_score	Referenced as FK in enforcement_queue, ontology_versions, exclusion_log; BEFORE INSERT checks cognitive_load_score < 0.70	No — system workflow
swat:ProvenanceEntity (PROV-O)	prov_entities	entity_iri VARCHAR(512) PK, activity_iri, agent_iri, generation_time, used_entity_iri, prov_json JSONB	Append-only; every functorial event writes PROV-O entity; linked via prov_o_ref columns across all tables	Read-only audit

11.6 Functor Health Monitoring — Semantic Drift Detection

The functor itself must be monitored for health. Semantic drift — where a constraint passes in PostgreSQL but would fail in the OWL reasoner, or vice versa — is a latent failure that can silently corrupt the system's governance integrity. The Functor Health Monitor runs continuous cross-validation between the two categories.

Drift Type	Detection Mechanism	Alert Threshold	Remediation
Verdict Divergence (most critical)	Automated 'shadow mode': every CB verdict is simultaneously evaluated by both Jena SHACL and Postgres triggers. Verdicts compared in real-time.	ANY divergence in verdict (pass vs. veto) for the same input vector triggers CRITICAL alert	Immediate: CB falls back to Jena SHACL as authoritative source. Postgres triggers suspended until functor re-validated. Incident created.
Threshold Drift	Daily automated test: insert known boundary-condition rows (e.g., S_a = 0.14999 and 0.15001) and verify Postgres CHECK constraints fire at same threshold as OWL	Any boundary test failure	Functor Synchronicity run: regenerate CHECK constraints from current SHACL shape; deploy via trustee-reviewed DDL migration
Unit Inconsistency	Weekly unit consistency audit: query all DECIMAL columns with unit tags; verify unit tags match OWL datatype annotations; verify no mixed-unit comparisons exist in triggers	Any mixed-unit comparison found in trigger code	Trigger rewrite: add explicit unit conversion step or raise unit_mismatch exception; update functor mapping documentation
Composition Order Break	Trigger fire-order test: log trigger execution sequence for a known SWRL chain. Compare against expected SWRL rule application order.	Any trigger fires out of expected cb_ON_order	Rename triggers to restore correct cb_ON_prefix order; re-run composition test matrix
PROV-O Linkage Gap	Referential integrity check: every compliance_events row must have a valid prov_o_ref linking to prov_entities. Weekly integrity scan.	> 0.1% rows missing prov_o_ref	Backfill PROV-O entities for affected rows; audit trigger code for missing provenance writes; alert if systematic gap found

11.7 Integration with End-to-End CB Phases

The Functorial Mapping Layer integrates directly into the eight-phase end-to-end flow specified in Section 4, providing the persistence backbone for the entire circuit breaker architecture:

CB Phase	Functorial Layer Role	Key Tables Written	Key Tables Read
Phase 0 (Design Time)	SHACL shapes and SWRL rules encoded into swat_triples; impossible_states table populated; functor validation test suite generated	swat_triples, impossible_states, swat_stages, ontology_versions	—
Phase 1 (SOP-01 Handshake)	Zone 2 SHACL constraints validated against initial sensor readings; morphism certificate written to prov_entities; trustee sign-off recorded	prov_entities, trustee_approvals, agent_cb_config	impossible_states, compliance_controls, ontology_versions
Phase 2 (Runtime Ingestion)	Sensor readings written to sensor_readings with ground_truth_flag; Zone 4 DECIMAL precision enforced; unit tags validated	sensor_readings	agent_cb_config (thresholds for comparison)
Phase 3 (Per-Agent Trust)	Zone 4 metrics computed and written to agent_trust_metrics; GENERATED k_trust column evaluated; Zone 1 SWRL triggers fire on actuator_commands	agent_trust_metrics, actuator_commands	sensor_readings, impossible_states, agent_cb_config
Phase 4 (Collective Composition)	Cross-stage mass balance check via sh:sparql Zone 2 mapping; D_s_total and D_b_total computed from agent_trust_metrics aggregation	compliance_events (if breach)	agent_trust_metrics (all 6 agents), sensor_readings (cross-stage FIT comparison)
Phase 5 (Attack Detection)	CB verdict written to enforcement_queue; Zone 1 SWRL trigger chain fires; all four Zones evaluated sequentially	enforcement_queue, compliance_events, prov_entities	agent_trust_metrics, impossible_states, compliance_controls
Phase 6 (Compliance/Freeze/Halt)	Zone 3 state transition guards enforce Tier 2/3 human-in-loop requirements; agent_status updated with BEFORE UPDATE trigger verification	enforcement_queue (status update), agent_status, trustee_approvals	compliance_controls, trustee_approvals (for Tier 2/3 gates)
Phase 7 (Recursive Governance)	SOP-03 exclusion_log populated; SOP-04 new	exclusion_log, swat_triples (ontology)	compliance_events (source of exclusion)

	impossible_states added via SME fold-back pipeline; ontology_versions incremented	updates), impossible_states	epoch), trustee_review_queue
Phase 8 (Human Teaming)	SME views provide real-time dashboard data; cognitive_load_score gates trustee_approvals; K_trust GENERATED column drives alert level	trustee_approvals (cognitive load check)	sme_cb_thresholds (view), sme_impossible_states (view), agent_trust_metrics

11.8 Functorial Architecture — System Map

The following table provides a consolidated view of the complete functorial mapping architecture, showing how each semantic construct in the OWL/SHACL/SWRL world maps to its PostgreSQL counterpart, and how the three pillars of the functorial guarantee are upheld across the full mapping.

Semantic Layer (C_OWL)	Functor F →	Relational Layer (C_Postgres)	Pillar	Drift Test Type
OWL Class	→	PostgreSQL Table	Validation	Schema completeness audit
OWL Individual	→	Table Row (with prov_o_ref)	Traceability	Row-to-IRI provenance chain
OWL Object Property	→	Foreign Key Reference	Validation	Referential integrity check
SHACL sh:minCount/maxCount	→	NOT NULL + UNIQUE constraint	Validation	Boundary insert test
SHACL sh:datatype + bounds	→	DECIMAL(p,s) + CHECK constraint	Validation	Threshold boundary test (0.14999/0.15000)
SHACL sh:class	→	FK to type table	Validation	Invalid type insert test
SHACL sh:sparql (computed)	→	Stored procedure + BEFORE trigger	Validation	Cross-stage mass balance test
SHACL sh:or/and/not	→	Trigger boolean expression	Validation	Boolean branch test matrix
SWRL Rule (leaf)	→	BEFORE INSERT trigger (cb_01_prefix)	Validation	Single-rule trigger test
SWRL Rule chain (A→B→C)	→	Trigger chain (cb_01_→cb_02_→cb_03_)	Validation	Composition order test; shadow mode

Compliance Control	→	State machine + BEFORE UPDATE guard	Traceability	State transition test; human gate test
Trust Metric (0–1 scale)	→	DECIMAL(6,5) with unit tag	Validation	Precision round-trip test; unit consistency audit
K_trust computation	→	GENERATED ALWAYS column	Validation	Known H_ai/L_h pairs → expected K_trust
PROV-O Entity	→	prov_entities row (append-only)	Traceability	PROV-O linkage gap scan (weekly)
Ontology version	→	ontology_versions row + CB config sync	Synchronicity	Version deployment propagation test
SHACL shape update	→	Functor-generated DDL migration	Synchronicity	SME edit fold-back round-trip test
SME relational view edit	→	Fold-back → triple store update → CB recert	Synchronicity	Full fold-back pipeline test